Assignment 2

ISM 6225 (Fall 2026) Assignment 2 Homework

Name	Mohammad Sahil	U#0	8679713

#1 – Subscription Revenue Calculator

Problem analysis

Element	Plan
Input	Current number of customers, monthly price per customer, percentage price increase
Data type	int for number of customers; decimal for monthly price and percentage because money should use decimal
Calculation	Current revenue = customers × current price. Price increase = percentage ÷ 100. New price = current price × (1 + increase rate). Projected revenue = customers × new price. Dollar increase = projected revenue − current revenue.
Decision	NO if/else decision is required because the same revenue calculations are performed for all valid inputs
Validation	Ensure the number of customers, monthly price, and percentage increases are valid numeric values and are not negative
Output	Current monthly revenue, projected monthly revenue, and dollar increase, displayed as currency with two decimal places

Final C Sharp code

```
Console.Write("Enter the current number of customers: ");
int customerCount = Convert.ToInt32(Console.ReadLine());

Console.Write("Enter the monthly price per customer: ");
decimal monthlyPrice = Convert.ToDecimal(Console.ReadLine());

Console.Write("Enter the percentage price increase: ");
decimal priceIncreasePercentage =
Convert.ToDecimal(Console.ReadLine());

decimal currentMonthlyRevenue = customerCount * monthlyPrice;
decimal projectedMonthlyRevenue = currentMonthlyRevenue * (1 +
priceIncreasePercentage / 100);
decimal monthlyRevenueIncrease = projectedMonthlyRevenue -
currentMonthlyRevenue;

Console.WriteLine($"Current monthly revenue:
{currentMonthlyRevenue:C2}”);
Console.WriteLine($"Projected monthly revenue:
{projectedMonthlyRevenue:C2}");
```

```
Console.WriteLine($"Increase in monthly revenue: {monthlyRevenueIncrease:C2}");
```

Code explanation

The program determines the impact of increased price on the monthly subscription income of the firm. To do so, the program requests the input of the current number of customers, monthly price per customer, and percentage increase in price. The number of customers is set to be an integer since it represents the actual count of customers. The monthly price and percentage increase are entered as decimals since they are used in the financial calculations. The program starts by computing the current monthly income by multiplying the number of customers and the monthly price. Then, the program changes the percentage increase to decimal rate by dividing it with 100 and computing the projected monthly income. The difference between the current monthly income and the projected monthly income is determined as an increase in income. Finally, the program displays the current income, projected income, and the income increase in currency format.

GenAI prompt record

Prompt	Exact prompt text	Purpose
Initial prompt	Create a simple C# console program for a Subscription Revenue Calculator. Ask the user for the current number of customers, monthly price per customer, and percentage price increase. Calculate the current monthly revenue, projected monthly revenue after the price increase, and the dollar increase. Use decimal for money and display all currency results to two decimal places with clear labels. Use only top-level C# statements, Console input/output, numeric variables, Convert methods, and arithmetic. Do not use LINQ, List<T>, custom classes, external packages, or concepts outside our course.	Requested an initial C# solution that follows the problem requirements and course constraints.
Follow-up prompt	Review the Question 1 code you generated. Check specifically whether decimal is used appropriately for money, whether the percentage price increase is correctly converted from a percentage to a decimal rate, and whether all currency outputs are formatted to two decimal places. Also check whether the required test with 1,250 customers, $39.95 monthly price, and a 7.5% increase produces the correct results. If you find any problem, explain it and provide the correction.	Checked the boundary, numeric type, and invalid-input behavior.

Evaluation and revision note

In terms of the calculations, the GenAI solution did use decimals for both the price per month and monthly revenue. In addition, the percentage increase was properly divided by 100. To validate the solution, I used test values of 1,250 customers, $39.95 for the monthly price, and 7.5 percent for the percentage increase. The calculations were correct as they matched the expected values, and the currency had proper two-digit format. With no calculation or formatting errors detected in the test, I decided not to change much about the logic of the solution.

Revision evidence

GenAI output or suggestion	Evaluation	Revision and reason
Used decimal for monthly price and revenue values.	Checked the code against the requirement that money should use decimal.	No change needed because decimal was correctly used for financial calculations.

GenAI output or suggestion	Evaluation	Revision and reason
Converted the percentage increase by dividing it by 100.	Tested the calculation using a 7.5% price increase and compared the result with the expected calculation.	No change needed because the percentage was converted correctly.
Used C2 formatting for revenue outputs.	Ran the program and verified that the results displayed as currency with two decimal places.	No change needed because the output formatting met the requirement.

Test evidence

Test type	Input	Expected result	Actual result	Status
10.5	1,250 customers, $39.95 price, 7.5% increase	Current: $49,937.50; Projected: $53,682.81; Increase: $3,745.31	Current: $49,937.50; Projected: $53,682.81; Increase: $3,745.31	Pass
Additional Test	100 customers, $20.00 price, 10% increase	Current: $2,000.00; Projected: $2,200.00; Increase: $200.00	Current: $2,000.00; Projected: $2,200.00; 'Increase: $200.00	Pass

Copied console output

```
Enter the current number of customers: 1250
Enter the monthly price per customer: 39.95
Enter the percentage price increase: 7.5
Current monthly revenue: $49,937.50
Projected monthly revenue: $53,682.81
Increase in monthly revenue: $3,745.31

Enter the current number of customers: 100
Enter the monthly price per customer: 20
Enter the percentage price increase: 10
Current monthly revenue: $2,000.00
Projected monthly revenue: $2,200.00
Increase in monthly revenue: $200.00
```

#2 – Investment Doubling Time

Problem analysis

Element	Plan
Input	Annual return rate entered by the user as a percentage
Data type	double for the annual return rate and doubling time because Math.Log works with double values
Calculation	Convert percentage to decimal: rate = percentage / 100. Calculate doubling time using Math.Log(2) / Math.Log(1 + rate)
Decision	If doubling time is less than 10 years → **Fast**; 10 through 20 years → **Moderate**; greater than 20 years → **Slow**
Validation	Annual return rate must be greater than zero. If it is zero or negative, display a clear error message and do not calculate doubling time.
Output	Doubling time displayed to two decimal places and its classification as Fast, Moderate, or Slow

Final C Sharp code

```
Console.Write("Enter the annual return rate as a percentage: ");
double annualReturnPercentage =
Convert.ToDouble(Console.ReadLine());

if (annualReturnPercentage <= 0)
{
    Console.WriteLine("The annual return rate must be greater
than zero.");
}
else
{
    double rate = annualReturnPercentage / 100;
    double doublingTime = Math.Log(2) / Math.Log(1 + rate);

    Console.WriteLine($"Investment doubling time:
{doublingTime:F2} years");
    if (doublingTime < 10)
    {
        Console.WriteLine("Classification: Fast");
    }
    else if (doublingTime <= 20)
    {
        Console.WriteLine("Classification: Moderate");
    }
    else
    {
        Console.WriteLine("Classification: Slow");
    }
}
```

Code explanation

The program determines how long it will take for an investment to double depending on the annual return rate. The program prompts the user to input the annual return rate expressed in percent and converts that rate into decimal by dividing it by 100. In case of zero or negative value, the program returns the error message, since the calculation must be done using positive return rate. In case of a positive value, the program uses the formula Math.Log(2) / Math.Log(1+rate) to perform the calculation. Math.Log is used to solve the equation for compound growth to find out how long it will take for the investment to double. The output value is formatted to display two decimal digits. Finally, the program classifies the doubling time as Fast in case it is less than 10 years, Moderate if it falls within 10-20 years range, and Slow if more than 20 years.

GenAI prompt record

Prompt	Exact prompt text	Purpose
Initial prompt	Create the C# code for Question 2 – Investment Doubling Time. First explain the formula log(2) / log(1 + rate), then generate the code. Ask the user for an annual return rate as a percentage greater than zero. Convert the percentage to a decimal rate and use Math.Log to calculate doubling time using log(2) / log(1 + rate). Display the result to two decimal places. Use if and else to classify the result as Fast if it is less than 10 years, Moderate if it is 10 through 20 years, or Slow if it is greater than 20 years. Reject a zero or negative rate with a clear message. Use only the C# concepts allowed in our assignment.	Requested an explanation of the formula and a C# solution following the Question 2 requirements and course constraints.
Follow-up prompt	Examine the code for Question 2 and see the classification boundary conditions. Explain what will happen if the doubled time comes out to be exactly 10 years and 20 years. Check if both these values have been classified as Moderate in accordance with the specifications.	Checked the classification logic at the exact 10-year and 20-year boundaries.

Evaluation and revision note

The correct implementation of the GenAI solution for computing the investment doubling period is by using Math.Log while converting the percent return rate into a decimal before performing the computation. I checked the conditions for the classification and found that doubling periods less than 10 years are classified as Fast, exactly between 10 and 20 years are Moderate, and more than 20 years are classified as Slow. I also found out that return rates of zero or below are rejected before computation.

Revision evidence

GenAI output or suggestion	Evaluation	Revision and reason
Used Math.Log(2) / Math.Log(1 + rate) to calculate doubling time.	Checked the formula against the Question 2 requirements.	No change needed because the correct formula was used.
Converted the percentage rate to decimal by dividing by `100`.	Checked that a value such as 7% becomes `0.07` before using the formula.	No change needed because the percentage conversion was correct.
Classified results using Fast, Moderate, and Slow.	Checked the conditions at the 10-year and 20-year boundaries.	No change needed because 10 through 20 years are correctly classified as Moderate.

GenAI output or suggestion	Evaluation	Revision and reason
Rejected zero or negative return rates.	Tested/reviewed the condition for a rate of 0%.	No change needed because invalid rates are prevented from entering the calculation.

Test evidence

Test type	Input	Expected result	Actual result	Status
Normal	Annual return rate = 7%	10.24 years, Moderate	10.24 years, Moderate	Pass
Normal	Annual return rate = 3%	23.45 years, Slow	23.45 years, Slow	Pass
Invalid input case	Annual return rate = 0%	Clear error message; doubling time is not calculated	Clear error message; doubling time is not calculated	Pass

Copied console output

```
Enter the annual return rate as a percentage: 7
Investment doubling time: 10.24 years
Classification: Moderate

Enter the annual return rate as a percentage: 3
Investment doubling time: 23.45 years
Classification: Slow

Enter the annual return rate as a percentage: 0
The annual return rate must be greater than zero.
```

#3 – Delivery Region Triangle

Problem analysis

Element	Plan
Input	Three triangle side lengths: `a`, `b`, and `c`
Data type	`double` for the three side lengths, semi-perimeter, and area because the values can contain decimals
Calculation	Semi-perimeter: s = (a + b + c) / 2; Area using Heron's formula: Math.Sqrt(s * (s-a) * (s-b) * (s-c))
Decision	Check that all sides are greater than `0`. Then check the triangle rules: `a + b > c`, `a + c > b`, and `b + c > a`
Validation	Use `try` and `catch` to handle nonnumeric input. Invalid side lengths or an invalid triangle must be rejected before `Math.Sqrt` is used
Output	For a valid triangle, display the calculated area. For invalid sides, an invalid triangle, or nonnumeric input, display an appropriate error message

Final C Sharp code

```
try
{
    Console.Write("Enter side length a: ");
    double a = Convert.ToDouble(Console.ReadLine());
    Console.Write("Enter side length b: ");
    double b = Convert.ToDouble(Console.ReadLine());
    Console.Write("Enter side length c: ");
    double c = Convert.ToDouble(Console.ReadLine());
    if (a > 0 && b > 0 && c > 0 && a + b > c && a + c > b && b +
c > a)
    {
        double semiPerimeter = (a + b + c) / 2;
        double area = Math.Sqrt(semiPerimeter * (semiPerimeter -
a) * (semiPerimeter - b) * (semiPerimeter - c));
        Console.WriteLine($"Triangle area: {area:F2}");
    }
    else
    {
        Console.WriteLine("Error: The side lengths must be
positive and satisfy the triangle rule.");
    }
}
catch (FormatException)
{
    Console.WriteLine("Error: Please enter numeric values for
all three side lengths.");
}
```

Code explanation

The program computes the area of the triangle representing a delivery region. In the program, the user will input three sides a, b, and c which are assigned double values so as to allow for the input of decimal measurements. The try and catch statement takes care of cases where the input values are not numeric so that the program does not crash. The program also performs some validations on the input data so as to confirm that the values entered for the sides are positive and that the three sides meet the requirement of forming a triangle, whereby the summation of any two sides is greater than the third one. This validation is done before Math.Sqrt operation to avoid computing an invalid value.

GenAI prompt record

Prompt	Exact prompt text	Purpose
Initial prompt	Create a C# console program for a purchase discount calculator. Ask for a purchase amount. Apply a 10% discount when the amount is $100 or more; otherwise apply no discount. Reject negative values and handle nonnumeric input. Display the original amount, discount, and final total as currency. Use only top-level statements, Console input and output, decimal variables, Convert.ToDecimal, try and catch, arithmetic, and if and else. Do not use classes, LINQ, collections, or external packages.	Requested an initial solution that followed the problem and course constraints.
Follow-up prompt	Review the code for two specific requirements: the discount must apply when the amount is exactly $100, and money must use decimal rather than double. Also explain how the program behaves when the user enters abc or a negative number. Return a corrected version if any requirement is not satisfied.	Checked the boundary, numeric type, and invalid-input behavior.

Evaluation and revision note

The GenAI implementation was accurate when it came to the try and catch block for non-numerical input and checking the triangle conditions before Math.Sqrt. The edge condition was checked using the values 1, 2, and 3 for the triangle sides and I could verify that the input is not accepted since the sides cannot create a triangle and Heron's formula does not calculate the square root of a negative number.

Revision evidence

GenAI output or suggestion	Evaluation performed	Revision and reason
Used try and catch for nonnumeric input.	Checked that entering text instead of a number produces an error message instead of crashing.	No change needed because nonnumeric input was handled correctly.
Checked all side lengths and triangle rules before using Math.Sqrt.	Tested the boundary input 1, 2, 3 and confirmed it is rejected as an invalid triangle.	No change needed because validation occurs before the area calculation.
Used Heron’s formula with Math.Sqrt for valid triangles.	Tested 3, 4, 5 and checked that the calculated area is 6	No change needed because the area calculation was correct.

Test evidence

Test type	Input	Expected result	Actual result	Status
Normal	a = 3, b = 4, c = 5	Valid triangle; Area = **6.00**	Valid triangle; Area = **6.00**	Pass
Boundary	a = 1, b = 2, c = 3	Invalid triangle; area should not be calculated	Invalid triangle message displayed; area not calculated	Pass
Invalid or alternative	a = abc	Nonnumeric input should be caught and an error message displayed	Invalid/nonnumeric input message displayed	Pass

Copied console output

```
Enter side length a: 3
Enter side length b: 4
Enter side length c: 5
Triangle area: 6.00
Enter side length a: 1
Enter side length b: 2
Enter side length c: 3
Error: The side lengths must be positive and satisfy the triangle rule.
Enter side length a: abc
Error: Please enter numeric values for all three side lengths.
```

#4 – Cloud Usage Invoice

Problem analysis

Element	Your analysis
Inputs	Number of compute hours used
Data types	double for compute hours and decimal for pricing rates, usage charge, service fee, and total invoice
Calculations	First 100 hours × `$0.12`; next 400 hours × `$0.09`; hours above 500 × `$0.06`. Service fee = usage charge × `0.05`. Total invoice = usage charge + service fee
Decisions	Use if, else if, and else to determine which progressive pricing tiers apply based on the number of hours
Validation	Check whether compute hours are negative. If hours are below `0`, display an error message and do not calculate the invoice
Outputs	Display the usage charge, 5% service fee, and total invoice with clear labels and currency formatting

Final C Sharp code

```
Console.Write("Enter compute hours: ");
decimal computeHours = Convert.ToDecimal(Console.ReadLine());

if (computeHours < 0)
{
```

```
    Console.WriteLine("Error: Compute hours cannot be
negative.");
}
else
{
    decimal usageCharge;

    if (computeHours <= 100)
    {
        usageCharge = computeHours * 0.12m;
    }
    else if (computeHours <= 500)
    {
        usageCharge = (100 * 0.12m) + ((computeHours - 100) *
0.09m);
    }
    else
    {
        usageCharge = (100 * 0.12m) + (400 * 0.09m) +
((computeHours - 500) * 0.06m);
    }

    decimal serviceFee = usageCharge * 0.05m;
    decimal totalInvoice = usageCharge + serviceFee;

    Console.WriteLine($"Usage charge: {usageCharge:C2}");
    Console.WriteLine($"Service fee: {serviceFee:C2}");
    Console.WriteLine($"Total invoice: {totalInvoice:C2}");
}
```

Code explanation

The program generates a monthly cloud usage invoice based on the amount of hours entered by the user. In case the hours entered are negative, an error message appears. Otherwise, for the valid input, the program will use if, else if, and else conditions to calculate the invoice using the principle of progressive pricing, where the first 100 hours cost $0.12 per hour, the next 400 hours cost $0.09 per hour, and any additional hours beyond 500 cost $0.06 per hour. With the principle of progressive pricing, the hours charged at each tier are only the ones in that tier rather than charging all the hours at one rate. Once the usage charge has been determined, the program will add a 5 percent service charge on it.

GenAI prompt record

Prompt	Exact prompt text	Purpose
Initial prompt	Create a simple C# console program for Question 4 – Cloud Usage Invoice. Ask the user for compute hours and reject negative values. Use progressive pricing: first 100 hours at $0.12 per hour, the next 400 hours at $0.09 per hour, and hours above 500 at $0.06 per hour. Use `if`, `else if`, and `else`. Add a 5% service fee and display the usage charge, service fee, and total invoice. Explain the difference between progressive pricing and flat pricing before generating the code. Use only the C# concepts allowed in our course.	Asked Copilot to create the program using progressive tier pricing instead of flat pricing.
Follow-up prompt 1	Trace the calculation for 700 compute hours step by step. Compare your calculation with the C# program and verify that each pricing tier is applied correctly. If there is an error, explain it and correct the code.	Checked that the program correctly calculates progressive pricing for the required 700-hour test.
Additional prompt	[Not used]	Not required

Evaluation and revision note

GenAI solution did not charge the same for all hours, but applied a progressive charging method as per the requirements. I tested the solution with the 700-hour requirement, and found that 100 hours, 400 hours, and the rest 200 hours have been charged at the correct prices. Further, I found that there was a 5% charge for the services, which is added after computing the price. There was no need for any major modification in the GenAI solution.

Revision evidence

GenAI output or suggestion	Evaluation performed	Revision and reason
Used progressive pricing for the three compute hour tiers.	Checked that only the hours within each tier were charged at that tier's rate.	No change needed because the progressive pricing logic was correct.
Used `if`, `else if`, and `else` to handle the pricing tiers.	Reviewed the conditions at the 100-hour and 500-hour boundaries.	No change needed because the tier conditions matched the requirements.
Calculated 700 hours as 100 × $0.12 + 400 × $0.09 + 200 × $0.06 = $60.00.	Manually calculated the 700-hour case and compared it with the program.	No change needed because the program produced the correct usage charge.
Added a 5% service fee after the usage charge.	Verified $60.00 × 0.05 = $3.00, giving a total invoice of $63.00.	No change needed because the service fee and total were calculated correctly.

Test evidence

Test type	Input	Expected result	Actual result	Status
Normal	80 hours	Error message displayed; invoice not calculated	Usage = **$9.60**, Fee = **$0.48**, Total = **$10.08**	Pass
Boundary	100 hours	Usage = **$12.00**, Fee = **$0.60**, Total = **$12.60**	Usage = **$12.00**, Fee = **$0.60**, Total = **$12.60**	Pass

Test type	Input	Expected result	Actual result	Status
Normal	350 hours	Usage = **$34.50**, Fee = **$1.73**, Total = **$36.23**	Usage = **$34.50**, Fee = **$1.73**, Total = **$36.23**	Pass
Boundary	500 hours	Usage = **$48.00**, Fee = **$2.40**, Total = **$50.40**	Usage = **$48.00**, Fee = **$2.40**, Total = **$50.40**	Pass
Normal/Above Tier	700 hours	Usage = **$60.00**, Fee = **$3.00**, Total = **$63.00**	Usage = **$60.00**, Fee = **$3.00**, Total = **$63.00**	Pass
Invalid	-1 hour	Error message; invoice should not be calculated	Error message displayed; invoice not calculated	Pass

Copied console output

```
Enter compute hours: 80
Usage charge: $9.60
Service fee: $0.48
Total invoice: $10.08

Enter compute hours: 100
Usage charge: $12.00
Service fee: $0.60
Total invoice: $12.60

Enter compute hours: 350
Usage charge: $34.50
Service fee: $1.73
Total invoice: $36.23

Enter compute hours: 500
Usage charge: $48.00
Service fee: $2.40
Total invoice: $50.40

Enter compute hours: 700
Usage charge: $60.00
Service fee: $3.00
Total invoice: $63.00

Enter compute hours: -1
Error: Compute hours cannot be negative.
```

#5 – Daily Sales Entry with a Sentinel

Problem analysis

Element	Your analysis
Inputs	Individual sales amounts entered one at a time. -1 is used to end data entry.
Data types	decimal for sales amounts, total, and average because they represent money; int for the number of valid sales
Calculations	Add each valid sale to the total and increase the valid sales counter. Average = total sales ÷ number of valid sales
Decisions	If input is -1, stop the loop. If input is another negative value, display an invalid-input message and continue. Otherwise, include it as a valid sale
Validation	-1 must not be included in the total or count. Other negative values are invalid and must also not be included
Repetition	Use a while loop to repeatedly ask for sales until the user enters -1
Outputs	Display the number of valid sales, total sales, and average sale. If no valid sales were entered, display an appropriate no-data message instead of calculating an average

Final C Sharp code

```
decimal saleAmount = 0m;
decimal totalSales = 0m;
int validSalesCount = 0;

while (saleAmount != -1)
{
    Console.Write("Enter a sale amount (-1 to finish): ");
    saleAmount = Convert.ToDecimal(Console.ReadLine());

    if (saleAmount == -1)
    {
        Console.WriteLine("Sales entry finished.");
    }
    else if (saleAmount < 0)
    {
        Console.WriteLine("Invalid sale amount. Enter a
nonnegative amount or -1 to finish.");
    }
    else
    {
        totalSales += saleAmount;
        validSalesCount++;
    }
}

if (validSalesCount > 0)
```

```
{
    decimal averageSale = totalSales / validSalesCount;
    Console.WriteLine($"Valid sales count: {validSalesCount}");
    Console.WriteLine($"Total sales: {totalSales:C2}");
    Console.WriteLine($"Average sale: {averageSale:C2}");
}
else
{
    Console.WriteLine("No valid sales were entered.");
}
```

Code explanation

The algorithm will store the amounts of sales made per day until -1 is entered by the user; the -1 will be taken as the sentinel value in order to terminate the process of data entry. In a while loop, the user will be prompted to enter the amount of the sale. Should the user enter the value -1, the loop terminates without including this value in the calculations. In the event that another negative value is entered, it will be assumed to be invalid and therefore not included in the computation. For each valid sale amount, the program will increment the total sales and valid sales counters. The program checks after the loop has terminated whether any valid sales have been entered. If there are valid sales entered, the average is calculated.

GenAI prompt record

Prompt	Exact prompt text	Purpose
Initial prompt	Create a simple C# console program for Question 5 – Daily Sales Entry with a Sentinel. Before writing the code, briefly describe the loop state needed, including the sale amount, total sales, and valid sales count. Then write the code using a `while` loop to repeatedly ask for a sale amount. Use `-1` as the sentinel to stop the loop and do not include `-1` in any calculation. Treat any other negative value as invalid and continue asking for input. Track the number of valid sales and total sales, then calculate the average. If no valid sales are entered, display a message instead of dividing by zero. Use only the C# concepts allowed in our course and do not use an array.	Asked Copilot to describe the loop state and create the program using a sentinel-controlled `while` loop.
Follow-up prompt 1	Review the Question 5 code and explain how it prevents `-1` from being included in the total or valid sales count. Also explain how the program avoids division by zero when `-1` is entered as the first input. If either condition is handled incorrectly, provide the correction.	Checked that the sentinel is excluded from calculations and that the no-data case does not cause division by zero.
Additional prompt	Not used	Not required

Evaluation and revision note

While loop correctly utilizes the while loop to repeatedly input the sale amount and -1 is used as the sentinel value. I made sure that -1 and any other negative number is not considered when summing up and counting the number of valid sales. I also made sure that when -1 is entered at the first prompt, the program will not do a divide by zero operation but will display a no-data message.

Revision evidence

GenAI output or suggestion	Evaluation performed	Revision and reason
Used a while loop to repeatedly collect sales.	Checked that the loop continues until the user enters -1.	No change needed because the loop correctly uses the sentinel to stop.
Used -1 as the sentinel.	Verified that -1 is not added to the total or valid sales count.	No change needed because the sentinel is excluded from calculations.
Rejected other negative sales values.	Tested -5 and confirmed it was treated as invalid without ending the loop.	No change needed because negative values other than -1 are handled correctly.
Checked the valid sales count before calculating the average.	Tested -1 as the first input.	No change needed because the program avoids division by zero and displays a no-data message.

Evaluation and revision note

As per the GenAI solution, the while loop was implemented and the value of -1 was considered as the sentinel. I checked that the value of -1 and negative numbers are not part of the sum and the valid number count. Further, I checked for the situation when -1 is entered as the first input and observed that the program does not divide by zero but gives a message indicating there is no data.

Test evidence

Test type	Input	Expected result	Actual result	Status
Normal/ Invalid mixed	120.50, 80, -5, 99.50, -1	`-5` rejected and `-1` ends input. Valid sales = **3**, Total = **$300.00**, Average = **$100.00**	Valid sales = **3**, Total = **$300.00**, Average = **$100.00**	Pass
Boundary/ No-data	`-1` as first input	Stop immediately. No valid sales; display a no-data message and do not calculate average	No valid sales message displayed; average not calculated	Pass

Copied console output

```
Enter a sale amount (-1 to finish): 120.50
Enter a sale amount (-1 to finish): 80
Enter a sale amount (-1 to finish): -5
Invalid sale amount. Enter a nonnegative amount or -1 to finish.
Enter a sale amount (-1 to finish): 99.50
Enter a sale amount (-1 to finish): -1
Sales entry finished.
Valid sales count: 3
Total sales: $300.00
Average sale: $100.00

Enter a sale amount (-1 to finish): -1
Sales entry finished.
No valid sales were entered.
```

#6 – Investment Payback Simulator

Problem analysis

Element	Your analysis
Inputs	Initial investment amount and expected monthly benefit
Data types	decimal for initial investment, monthly benefit, cumulative benefits, and unrecovered balance because these are money values; int for the month counter
Calculations	Add the monthly benefit to cumulative benefits each month. Unrecovered balance = initial investment − cumulative benefits
Decisions	Check that both inputs are greater than 0. Stop when cumulative benefits are greater than or equal to the initial investment or when 600 months are reached
Validation	Reject zero or negative initial investment or monthly benefit
Repetition	Use a while loop to simulate each month until the investment is recovered, with a maximum of 600 months
Outputs	Display the unrecovered balance after each month, the payback month, and cumulative benefits at payback

Final C Sharp code

```
Console.Write("Enter the initial investment: ");
decimal initialInvestment =
Convert.ToDecimal(Console.ReadLine());

Console.Write("Enter the monthly benefit: ");
decimal monthlyBenefit = Convert.ToDecimal(Console.ReadLine());

if (initialInvestment <= 0 || monthlyBenefit <= 0)
{
    Console.WriteLine("Error: The initial investment and monthly
benefit must both be positive.");
}
else
{
    decimal cumulativeBenefits = 0m;
    int month = 0;
    int maximumMonths = 600;

    while (cumulativeBenefits < initialInvestment && month <
maximumMonths)
    {
        month++;
        cumulativeBenefits += monthlyBenefit;
```

```
        decimal unrecoveredBalance = initialInvestment -
cumulativeBenefits;
        if (unrecoveredBalance < 0)
        {
            unrecoveredBalance = 0;
        }

        Console.WriteLine($"Month {month}: Unrecovered balance:
{unrecoveredBalance:C2}");
    }

    if (cumulativeBenefits >= initialInvestment)
    {
        Console.WriteLine($"Payback month: {month}");
        Console.WriteLine($"Cumulative benefits at payback:
{cumulativeBenefits:C2}");
    }
    else
    {
        Console.WriteLine($"Investment was not recovered within
{maximumMonths} months.");
        Console.WriteLine($"Cumulative benefits:
{cumulativeBenefits:C2}");
    }
}
```

Code explanation

The program computes how long it will take for an initial investment to be paid back based on a constant monthly benefit. The program first requires the user to input the amount of the initial investment as well as the expected monthly benefit. The program ensures that both the initial investment and the monthly benefit are positive. A while loop is employed for computing the payback of the investment. This involves adding the monthly benefit to the total benefits every month. The month count is incremented by one in every loop as well as calculating and displaying the amount still left to be paid back. The process continues until the total benefits are equal to or more than the initial investment. There is a maximum of 600 months incorporated in the loop as a safeguard.

GenAI prompt record

Prompt	Exact prompt text	Purpose
Initial prompt	Create a simple C# console program for Question 6 – Investment Payback Simulator. First provide simple pseudocode, then generate the C# code. Ask the user for a positive initial investment and a positive monthly benefit. Reject zero or negative inputs. Use a `while` loop to add the monthly benefit to cumulative benefits each month until the initial investment is recovered. Count the months and display the unrecovered balance after each month. When the investment is recovered, display the payback month and cumulative benefits at payback. Add a maximum limit of 600 months as a defensive stopping condition. Use only the C# concepts allowed in our	Asked GenAI to first develop pseudocode and then create the C# solution using the required `while` loop and defensive limit

Prompt	Exact prompt text	Purpose
Follow-up prompt 1	Review the Question 6 code for possible off-by-one errors in the month counter. Test an initial investment of $9,000 with a monthly benefit of $3,000 and confirm that the payback month is exactly month 3. Also verify that the 600-month defensive limit works correctly. If there is an error, explain it and provide the correction.	[Checked the month counter, exact-month payback boundary, and 600-month stopping condition.
Additional prompt	Not used	No additional prompt was needed.

Evaluation and revision note

The GenAI response used a while loop to calculate the months required for the return on investment as intended. I examined the month calculation to look for an off-by-one mistake, and I tested the scenario where the payoff is $9,000, but the gain per month is $3,000, and it correctly calculated the payoff period as 3 months. The 600-month safeguard is also included in the program.

Revision evidence

GenAI output or suggestion	Evaluation performed	Revision and reason
Used a while loop to add the monthly benefit until the investment was recovered	Tested $9,000 initial investment with $3,000 monthly benefit and checked the month counter	Kept the loop because it correctly reported payback at month 3
Suggested displaying the unrecovered balance after every month	after each loop iteration to make sure the balance decreased correctly	requirement to show the unrecovered balance each month
Used a 600-month maximum stopping condition	Reviewed the loop condition to make sure it could not continue beyond 600 simulated months	Kept the defensive limit because it prevents an excessively long simulation
Month counting could create an off-by-one error if the counter were updated in the wrong place	Traced the $9,000 / $3,000 case month by month: Month 1 = $3,000, Month 2 = $6,000, Month 3 = $9,000	Confirmed the month counter should increase when each monthly benefit is added so the exact payback month is reported correctly

Test evidence

Test type	Input	Expected result	Actual result	Status
Normal	Initial investment = `$10,000`, Monthly benefit = `$1,800`	Payback in **6 months**; cumulative benefits at payback = **$10,800.00**	Payback month = **6**; cumulative benefits = **$10,800.00**	Pass
Boundary	initial investment = $9,000, Monthly benefit = $3,000	Payback in exactly **3 months**; cumulative benefits = **$9,000.00**; unrecovered balance = **$0.00**	Payback month = **3**; cumulative benefits = **$9,000.00**; final unrecovered balance = **$0.00**	Pass
Invalid or alternative	Initial investment = -1000, Monthly benefit = $500	Display an invalid-input message; payback simulation should not run	Error: The initial investment and monthly benefit must both be positive.	Pass

Copied console output

```
Enter the monthly benefit: 1800
Month 1: Unrecovered balance: $8,200.00
Month 2: Unrecovered balance: $6,400.00
Month 3: Unrecovered balance: $4,600.00
Month 4: Unrecovered balance: $2,800.00
Month 5: Unrecovered balance: $1,000.00
Month 6: Unrecovered balance: $0.00
Payback month: 6
Cumulative benefits at payback: $10,800.00

Enter the initial investment: 9000
Enter the monthly benefit: 3000
Month 1: Unrecovered balance: $6,000.00
Month 2: Unrecovered balance: $3,000.00
Month 3: Unrecovered balance: $0.00
Payback month: 3
Cumulative benefits at payback: $9,000.00

Enter the initial investment: -1000
Enter the monthly benefit: 500
Error: The initial investment and monthly benefit must both be positive.
```

#7 – Weekly Demand Bar Chart

Problem analysis

Element	Plan
Input	Seven demand values entered by the user, one for each day. Each value must be zero or greater.
Data type	An `int` array with seven elements stores the daily demand values because demand is entered as whole numbers. Integer variables are used for the loop counters.
Calculation	Divide each demand value by `10` using integer division to determine the number of asterisks. Any remainder below 10 is ignored.
Decision	Check whether each entered demand value is zero or greater. If a value is negative, reject it and request a valid value.
Validation	Use one loop to populate the seven-element array and another loop to display the chart. Use a nested loop to print one `*`for every 10 units of demand.
Output	Display the day number, demand value, and the corresponding number of asterisks for each of the seven days.

Final C Sharp code

```
int[] dailyDemand = new int[7];

for (int day = 0; day < 7; day++)
{
    int demand;
    do
    {
        Console.Write($"Enter demand for day {day + 1}: ");
        demand = Convert.ToInt32(Console.ReadLine());

        if (demand < 0)
        {
            Console.WriteLine("Demand must be zero or greater.
Please try again.");
        }
    } while (demand < 0);

    dailyDemand[day] = demand;
}

for (int day = 0; day < 7; day++)
{
    Console.Write($"Day {day + 1}: Demand {dailyDemand[day]} ");

    for (int star = 0; star < dailyDemand[day] / 10; star++)
    {
```

```
            Console.Write("*");
        }

        Console.WriteLine();
    }
```

Code explanation

The seven daily demand figures are saved in an array of integers, each element representing a day. A for loop is used to enter the demand values into the program. The program checks that the entered values are either zero or positive because negative demands are not allowed. After entering the seven figures, the program runs another for loop where it displays both the day and the demand. To find the number of stars to be printed for each day, the computer divides the integer demand by ten. In this way, it only considers the number without considering the remainder less than ten. The number of stars is printed using a nested loop printing * one after another rather than repeating strings.

GenAI prompt record

Prompt	Exact prompt text	Purpose
Initial prompt	Create a simple C# console program that stores 7 daily demand values in an integer array. Each value must be zero or greater. Use one loop to enter the values and another loop to display the results. For each day, show the day number, demand value, and one * for every 10 units using a nested loop. Ignore remainders below 10. Do not use string repetition, LINQ, or List.	Requested a solution using arrays and loops while following the program requirements.
Follow-up prompt	Trace the array indexes and nested-loop iteration counts using these demand values: 0, 9, 10, 24, 50, 73, 100. Check that the program prints 0, 0, 1, 2, 5, 7, and 10 asterisks for the seven days. If there is an error, explain it and correct the code.	Checked the array indexes, integer division, and number of nested-loop iterations for the required test values.

Evaluation and revision note

The GenAI solution used the right approach in employing an integer array and the right loops for inputting and printing out the 7 values for the demands. I have tested the nested loops approach with the necessary values of 0, 9, 10, 24, 50, 73, and 100. The values with remainders like 24 and 73 required integer division so that the remainder is not considered. I accepted the demand/10 approach since it yields the correct number of stars as 2 for 24 and 7 for 73.

Revision evidence

GenAI output or suggestion	Evaluation	Revision and reason
Used an integer array to store the seven demand values	Checked that each array index stores one day's demand	Kept the integer array because it correctly stores all seven daily values
Used `demand / 10` to determine the number of asterisks	Tested `0, 9, 10, 24, 50, 73, 100` and checked the expected star counts	Kept integer division because it correctly ignores remainders below 10
Used a nested loop to print the asterisks	Checked that 24 produces 2 stars and 73 produces 7 stars	Kept the nested loop because it prints one `*` for every 10 units without using string repetition

Test evidence

Test type	Input	Expected result	Actual result	Status
Normal	0, 9, 10, 24, 50, 73, 100	Stars for Days 1–7: `0, 0, 1, 2, 5, 7, 10`	Days 1–7 displayed 0, 0, 1, 2, 5, 7, 10 stars	Pass
Boundary	`0, 9, 10` within the required test data	`0` → 0 stars; `9` → 0 stars; `10` → 1 star	`0` and `9` displayed no stars; `10` displayed exactly 1 star	Pass
Invalid or Alternative	Negative value such as `-5`	Reject `-5` and ask for Day 1 again; valid values should produce `2, 3, 4, 5, 6, 7, 8` stars	`-5` was rejected with a message and Day 1 was requested again; valid values displayed `2, 3, 4, 5, 6, 7, 8` stars	Pass

Copied console output

```
Enter demand for day 1: 0
Enter demand for day 2: 9
Enter demand for day 3: 10
Enter demand for day 4: 24
Enter demand for day 5: 50
Enter demand for day 6: 73
Enter demand for day 7: 100
Day 1: Demand 0
Day 2: Demand 9
Day 3: Demand 10 *
Day 4: Demand 24 **
Day 5: Demand 50 *****
Day 6: Demand 73 *******
Day 7: Demand 100 **********Enter demand for day 1: 0
Enter demand for day 2: 9
Enter demand for day 3: 10
```

```
Enter demand for day 4: 0
Enter demand for day 5: 9
Enter demand for day 6: 10
Enter demand for day 7: 0
Day 1: Demand 0
Day 2: Demand 9
Day 3: Demand 10 *
Day 4: Demand 0
Day 5: Demand 9
Day 6: Demand 10 *
Day 7: Demand 0

Enter demand for day 1: -5
Demand must be zero or greater. Please try again.
Enter demand for day 1: 20
Enter demand for day 2: 30
Enter demand for day 3: 40
Enter demand for day 4: 50
Enter demand for day 5: 60
Enter demand for day 6: 70
Enter demand for day 7: 80
Day 1: Demand 20 **
Day 2: Demand 30 ***
Day 3: Demand 40 ****
Day 4: Demand 50 *****
Day 5: Demand 60 ******
Day 6: Demand 70 *******
Day 7: Demand 80 ********
```

#8 – Customer Risk Score Method

Problem analysis

Element	Plan
Input	Number of missed payments, support tickets, and months inactive entered by the user. All values must be zero or greater.
Data type	Use int for missed payments, support tickets, months inactive, and the risk score because all values are whole numbers.
Calculation	Calculate the risk score using: missedPayments × 20 + supportTickets × 5 + monthsInactive × 8. The calculation is performed inside the ComputeRiskScore method.
Decision	Classify the score as **Low** when it is less than 30, **Moderate** when it is 30–59, and **High** when it is 60 or greater. Reject negative input values.
Validation	Use one loop to populate the seven-element array and another loop to display the chart. Use a nested loop to print one `*`for every 10 units of demand.
Output	Display the calculated risk score and its classification as Low, Moderate, or High.

Final C Sharp code

```
static int ComputeRiskScore(int missedPayments, int
supportTickets, int monthsInactive)
{
    int riskScore = (missedPayments * 20) + (supportTickets * 5)
+ (monthsInactive * 8);
    return riskScore;
}

Console.Write("Enter missed payments: ");
int missedPayments = Convert.ToInt32(Console.ReadLine());

Console.Write("Enter support tickets: ");
int supportTickets = Convert.ToInt32(Console.ReadLine());

Console.Write("Enter months inactive: ");
int monthsInactive = Convert.ToInt32(Console.ReadLine());

if (missedPayments < 0 || supportTickets < 0 || monthsInactive <
0)
{
    Console.WriteLine("Error: All values must be zero or
greater.");
}
else
{
    int riskScore = ComputeRiskScore(missedPayments,
supportTickets, monthsInactive);
    string classification;

    if (riskScore < 30)
    {
        classification = "Low";
    }
    else if (riskScore >= 30 && riskScore <= 59)
    {
        classification = "Moderate";
    }
    else
    {
        classification = "High";
    }

    Console.WriteLine($"Risk score: {riskScore}");
```

```
    Console.WriteLine($"Classification: {classification}");
}
```

Code explanation

This program will compute the customer risk score by using three variables: missed payments, support tickets, and months inactive. All these variables will be declared as an integer because they can only accept a whole number as their value. All the variables will be validated to be zero or above before computing the score. In order to calculate the customer's risk score, the ComputeRiskScore function will be used. This function will use the three values as its parameters and compute the risk score by multiplying missed payments with 20, support tickets with 5, and months inactive with 8. It will return the computed integer score back to the main program. The classification will remain outside the function. If statement along with else if statement will classify the score lower than 30 as low, between 30 and 59 as moderate, and more than 60 as high.

GenAI prompt record

Prompt	Exact prompt text	Purpose
Initial prompt	Create a simple C# console program that asks the user for missed payments, support tickets, and months inactive. All inputs must be zero or greater. Create a method named ComputeRiskScore that accepts these three integer values and returns an integer score using: missed payments × 20 + support tickets × 5 + months inactive × 8. Keep the score calculation inside the method and the classification outside the method. Classify the score as Low if below 30, Moderate if 30–59, and High if 60 or greater. Display the risk score and classification. Use only simple C# concepts such as methods, integers, Console input/output, arithmetic, and if/else	Requested a solution that separates the risk-score calculation into a method and keeps the classification logic outside the method.
Follow-up prompt	Review the classification boundaries in the code. Check scores of 29, 30, 59, and 60 and explain how each one is classified. Confirm that 29 is Low, 30 and 59 are Moderate, and 60 is High. If any boundary is incorrect, provide the corrected condition.	Checked the Low, Moderate, and High classification boundaries for possible errors.

Evaluation and revision note

GenAI's approach has appropriately divided the Risk Score calculation to the ComputeRiskScore procedure and has not included any classification in the ComputeRiskScore procedure. I analyzed the boundary conditions and determined that it was necessary to include comparisons that would allow proper handling of scores equal to 30 and 60. I confirmed that scores less than 30 are Low, scores between 30 and 59 are Moderate, and scores above 60 are High.

Revision evidence

GenAI output or suggestion	Evaluation	Revision and reason
Placed the risk score calculation inside the `ComputeRiskScore` method	Checked that the method accepts the three required integer parameters and returns the calculated integer score	Kept the method because it correctly separates score calculation from classification
Used if and else if to classify the risk score	Checked the boundary scores 29, 30, 59, and 60	Confirmed the conditions so 29 is Low, 30–59 is Moderate, and 60 or greater is High
Kept the classification logic outside the ComputeRiskScore method	Reviewed the method to make sure it only calculates and returns the score	Kept the classification outside the method because the requirement separates calculation from classification

Test evidence

Test type	Input	Expected result	Actual result	Status
Normal	Missed payments = 0, Support tickets = 1, Months inactive = 0	Risk score = **5**; Classification = **Low**	Risk score = **5**; Classification = **Low**	Pass
Normal	Missed payments = 1, Support tickets = 2, Months inactive = 0	Risk score = **30**; Classification = **Moderate**	Risk score = **30**; Classification = **Moderate**	Pass
Normal	Missed payments = 2, Support tickets = 1, Months inactive = 2	Risk score = **61**; Classification = **High**	Risk score = **61**; Classification = **High**	Pass
Boundary	Missed payments = 3, Support tickets = 0, Months inactive = 0	Risk score = **60**; Classification = **High**	Risk score = **60**; Classification = **High**	Pass

Copied console output

```
Enter missed payments: 0
Enter support tickets: 1
Enter months inactive: 0
Risk score: 5
Classification: Low

Enter missed payments: 1
Enter support tickets: 2
Enter months inactive: 0
Risk score: 30
Classification: Moderate

Enter missed payments: 2
Enter support tickets: 1
Enter months inactive: 2
Risk score: 61
Classification: High

Enter missed payments: 3
```

```
Enter support tickets: 0
Enter months inactive: 0
Risk score: 60
Classification: High
```

#9 – Transaction Anomaly Screening

Problem analysis

Element	Plan
Input	Number of transactions from 3 through 20, followed by each transaction amount. Transaction amounts must be zero or greater.
Data type	int for the number of transactions and loop counters; decimal array for transaction amounts; decimal for minimum, maximum, total, mean, and threshold.
Calculation	Calculate total, minimum, maximum, mean = total / number of transactions, and review threshold = mean × 1.5.
Decision	Reject a transaction count outside 3–20 and reject negative transaction amounts. Label an amount **Review** if it is greater than 1.5 times the mean; otherwise label it **Normal**.
Repetition	Use loops to enter transaction amounts and calculate statistics. Use a second pass through the array to classify every transaction.
Output	Minimum, maximum, total, mean, and each transaction amount with its **Normal** or **Review** label.

Final C Sharp code

```
Console.Write("How many transaction amounts will be entered? (3
to 20): ");
int count = Convert.ToInt32(Console.ReadLine());

while (count < 3 || count > 20)
{
    Console.WriteLine("Please enter a number from 3 to 20.");
    Console.Write("How many transaction amounts will be entered?
(3 to 20): ");
    count = Convert.ToInt32(Console.ReadLine());
```

```
}

decimal[] transactions = new decimal[count];

for (int i = 0; i < count; i++)
{
    decimal amount;

    do
    {
        Console.Write($"Enter transaction {i + 1}: ");
        amount = Convert.ToDecimal(Console.ReadLine());

        if (amount < 0)
        {
            Console.WriteLine("Transaction amount cannot be
negative. Please try again.");
        }
    } while (amount < 0);

    transactions[i] = amount;
}

decimal minValue = transactions[0];
decimal maxValue = transactions[0];
decimal total = 0m;

for (int i = 0; i < count; i++)
{
    if (transactions[i] < minValue)
    {
        minValue = transactions[i];
    }

    if (transactions[i] > maxValue)
    {
        maxValue = transactions[i];
    }

    total += transactions[i];
}

decimal mean = total / count;

Console.WriteLine($"Minimum: {minValue:C2}");
Console.WriteLine($"Maximum: {maxValue:C2}");
```

```
Console.WriteLine($"Total: {total:C2}");
Console.WriteLine($"Mean: {mean:C2}");

for (int i = 0; i < count; i++)
{
    string label;

    if (transactions[i] > mean * 1.5m)
    {
        label = "Review";
    }
    else
    {
        label = "Normal";
    }

    Console.WriteLine($"Transaction {i + 1}:
{transactions[i]:C2} - {label}");
}
```

Code explanation

The transactions are stored in a decimal array since the transaction amount can consist of decimal figures. The program uses loops to enter the transaction amounts and ensure that negative numbers are not allowed. Another loop is utilized to calculate the sum, minimum, and maximum but not using LINQ or aggregate methods. The initial minimum and maximum amounts are set using the first transaction amount so that the calculation begins with the real data value. To calculate the mean, the program divides the sum by the number of transactions. Once the mean is known, the array must be looped again to evaluate the transaction amounts. In this case, the threshold will be 1.5 times the mean. Any transaction greater than this threshold will be set as Review while the rest will be Normal. A disadvantage of this approach is that a high transaction amount may lead to a higher mean and threshold value.

GenAI prompt record

Prompt	Exact prompt text	Purpose
Initial prompt	Create a simple C# console program that asks how many transaction amounts will be entered, from 3 through 20. Store the transaction amounts in a decimal array and reject negative values. Use loops to calculate the minimum, maximum, total, and mean. Do not use LINQ or built-in Min, Max, Sum, or Average methods. Then use a second pass through the array to label each transaction as Review if it is greater than 1.5 times the mean; otherwise label it Normal. Display the minimum, maximum, total, mean, and the label for every transaction. Use only simple C# concepts allowed in the assignment.	Requested a loop-based solution using arrays without LINQ or built-in aggregate methods.
Follow-up prompt	Check how the minimum and maximum values are initialized in the program. Explain why initializing them from the first transaction is better than starting them at zero. Also explain why two passes through the transaction array are needed to correctly label each transaction as Normal or Review.	Checked the minimum and maximum initialization and verified why the screening requires two passes through the array.

Evaluation and revision note

GenAI solution had properly employed the use of decimal array along with loops in order to find out the minimum, maximum, sum, and mean without using LINQ. In the review process, I have realized that initialization of minimum and maximum is likely to give wrong output if the values are just assigned the value of zero; thus, I have ensured that the minimum and maximum need to be initialized to the first transaction value. The solution also requires two iterations since mean needs to be found out before comparison with 1.5 times the mean.

Revision evidence

GenAI output or suggestion	Evaluation	Revision and reason
Used a decimal array to store the transaction amounts	Checked that all entered amounts were stored and could be processed using loops	Kept the decimal array because it correctly stores transaction amounts and follows the assignment requirements
Initialized the minimum and maximum using the first transaction	Compared this approach with initializing the values to zero	Kept the first transaction as the initial minimum and maximum because it provides a valid starting value from the dataset
Used a second pass to classify each transaction	Checked whether the Review threshold could be applied before calculating the mean	Kept the second pass because the mean must be known before comparing each transaction with 1.5 × mean

Test evidence

Test type	Input	Expected result	Actual result	Status
Required test	100, 110, 95, 105, 500	Min = **95**, Max = **500**, Total = **910**, Mean = **182**. First four transactions = **Normal**; 500 = **Review**	Min = **95**, Max = **500**, Total = **910**, Mean = **182**. First four transactions = **Normal**; 500 = **Review**	Pass
No Review test	100, 110, 95, 105, 120	Min = **95**, Max = **120**, Total = **530**, Mean = **106**. All transactions = **Normal**	Min = **95**, Max = **120**, Total = **530**, Mean = **106**. All transactions = **Normal**	Pass

Copied console output

```
How many transaction amounts will be entered? (3 to 20): 5
Enter transaction 1: 100
Enter transaction 2: 110
Enter transaction 3: 95
Enter transaction 4: 105
Enter transaction 5: 500
Minimum: $95.00
Maximum: $500.00
Total: $910.00
Mean: $182.00
Transaction 1: $100.00 - Normal
Transaction 2: $110.00 - Normal
Transaction 3: $95.00 - Normal
Transaction 4: $105.00 - Normal
Transaction 5: $500.00 - Review

How many transaction amounts will be entered? (3 to 20): 5
Enter transaction 1: 100
Enter transaction 2: 110
Enter transaction 3: 95
Enter transaction 4: 105
Enter transaction 5: 120
Minimum: $95.00
Maximum: $120.00
```

```
Total: $530.00
Mean: $106.00
Transaction 1: $100.00 - Normal
Transaction 2: $110.00 - Normal
Transaction 3: $95.00 - Normal
Transaction 4: $105.00 - Normal
Transaction 5: $120.00 - Normal
```

#10 – Seven Day Performance Summary

Problem analysis

Element	Your analysis
Inputs	Seven daily sales values. Each sales value must be numeric and zero or greater.
Data types	decimal array for the seven sales values, decimal for total, average, highest, and lowest sales, and int for loop counters and number of days above average.
Calculations	Calculate total sales, average sales, highest sales, lowest sales, and number of days above average using separate methods.
Decisions	Reject invalid or negative sales values. Classify the week as **Strong** when at least 4 days are above average, **Balanced**when 2 or 3 days are above average, and **Concentrated** when fewer than 2 days are above average.
Repetition	Use loops to collect the seven sales values, display the daily list, and perform calculations inside the methods.
Outputs	Display each day's sales, total sales, average sales, highest sales, lowest sales, number of days above average, and the weekly classification.

Final C Sharp code

```
static decimal CalculateTotal(decimal[] sales)
{
    decimal total = 0m;
    for (int i = 0; i < sales.Length; i++)
    {
        total += sales[i];
    }
    return total;
}
```

```
static decimal CalculateAverage(decimal[] sales)
{
    decimal total = 0m;
    for (int i = 0; i < sales.Length; i++)
    {
        total += sales[i];
    }
    return total / sales.Length;
}
```

```
static decimal CalculateHighest(decimal[] sales)
{
    decimal highest = sales[0];
    for (int i = 1; i < sales.Length; i++)
    {
        if (sales[i] > highest)
        {
            highest = sales[i];
        }
    }
    return highest;
}

static decimal CalculateLowest(decimal[] sales)
{
    decimal lowest = sales[0];
    for (int i = 1; i < sales.Length; i++)
    {
        if (sales[i] < lowest)
        {
            lowest = sales[i];
        }
    }
    return lowest;
}

static int CountAboveAverage(decimal[] sales)
{
    decimal total = 0m;
    for (int i = 0; i < sales.Length; i++)
    {
        total += sales[i];
    }
    decimal average = total / sales.Length;
    int count = 0;
    for (int i = 0; i < sales.Length; i++)
    {
        if (sales[i] > average)
        {
            count++;
        }
    }
    return count;
}

decimal[] salesForWeek = new decimal[7];
```

```
for (int i = 0; i < salesForWeek.Length; i++)
{
    while (true)
    {
        try
        {
            Console.Write($"Enter sales for day {i + 1}: ");
            decimal salesValue =
Convert.ToDecimal(Console.ReadLine());

            if (salesValue < 0)
            {
                Console.WriteLine("Sales must be zero or
greater. Please try again.");
                continue;
            }

            salesForWeek[i] = salesValue;
            break;
        }
        catch (FormatException)
        {
            Console.WriteLine("Please enter a valid number.");
        }
    }
}

Console.WriteLine();
Console.WriteLine("Daily sales values:");
for (int i = 0; i < salesForWeek.Length; i++)
{
    Console.WriteLine($"Day {i + 1}: {salesForWeek[i]:C2}");
}

decimal totalSales = CalculateTotal(salesForWeek);
decimal averageSales = CalculateAverage(salesForWeek);
decimal highestSales = CalculateHighest(salesForWeek);
decimal lowestSales = CalculateLowest(salesForWeek);
int daysAboveAverage = CountAboveAverage(salesForWeek);

Console.WriteLine();
Console.WriteLine($"Total sales: {totalSales:C2}");
Console.WriteLine($"Average sales: {averageSales:C2}");
Console.WriteLine($"Highest sales: {highestSales:C2}");
Console.WriteLine($"Lowest sales: {lowestSales:C2}");
```

```
Console.WriteLine($"Number of days above average:
{daysAboveAverage}");

if (daysAboveAverage >= 4)
{
    Console.WriteLine("Week classification: Strong");
}
else if (daysAboveAverage >= 2)
{
    Console.WriteLine("Week classification: Balanced");
}
else
{
    Console.WriteLine("Week classification: Concentrated");
}
```

Code explanation

The program uses a decimal array to store seven values of daily sales and ensures that all the values are not negative. A try/catch block is used for nonnumeric input in order to allow the user to enter a valid value rather than terminate the program. Different methods are defined for calculating total sales, average sales, maximum sales, minimum sales, and days above average sales. An array is used in all these methods, and loops are used to perform calculations on array values without using LINQ and built-in aggregate functions. The program prints daily sales values and five summary statistics in the main part of the program. Finally, days above average are used to classify the week as Strong (at least four days above average), Balanced (two or three days above average), or Concentrated (fewer than two days above average).

GenAI prompt record

Prompt	Exact prompt text	Purpose
Initial prompt	Design a simple C# console program that collects seven nonnegative decimal sales values in an array and handles nonnumeric input. The program must use separate methods to calculate total sales, average sales, highest sales, lowest sales, and the number of days above average. Each calculation method must use loops and cannot use LINQ or built-in aggregate methods. The program should display the seven daily sales values and all five summary measures. It should classify the week as Strong when at least four days are above average, Balanced when two or three days are above average, or Concentrated when fewer than two days are above average. Explain the program design, method responsibilities, input validation, and overall flow only. Do not write the C# code yet.	Requested the program design and method responsibilities before generating code.
Follow-up prompt	Create a simple C# console program that asks the user to enter seven nonnegative decimal sales values and stores them in an array. Use try/catch for nonnumeric input. Create separate methods using loops to calculate total sales, average sales, highest sales, lowest sales, and the number of days above average. Do not use LINQ or built-in Sum, Average, Max, or Min methods. Display each day's sales and all five results. Classify the week as Strong if at least 4 days are above average, Balanced if 2 or 3 days are above average, or Concentrated if fewer than 2 days are above average. Keep the code simple and use only basic C# concepts.	Requested the C# implementation after reviewing the program design.

Evaluation and revision note

The GenAI code has done the right thing by dividing the program into methods of calculating total, average, max, min, and days above average. While reviewing this program, I found out that it needs to consider non-numeric and negative sales as well without terminating the process or storing any such invalid data. The try/catch validation was left intact and it was checked whether there were any loops for calculating the averages or if the LINQ/aggregate method was used.

Revision evidence

GenAI output or suggestion	Evaluation	Revision and reason
Used separate methods for total, average, highest, lowest, and days above average	Reviewed each method to confirm that it performs one calculation and uses loops	Kept the separate methods because they clearly divide the program responsibilities
Used try/catch and validation for sales input	Checked how the program responds to nonnumeric and negative values	Kept the validation so invalid values are rejected and the user can enter a valid correction
Used the number of days above average to classify the week	Checked the conditions for Strong, Balanced, and Concentrated	Strong, Balanced, and ConcentratedKept the conditions because they correctly classify 4 or more as Strong, 2–3 as Balanced, and fewer than 2 as Concentrated
Avoided LINQ and built-in aggregate methods	Reviewed the calculation methods for Sum, Average, Max, Min, or LINQ usage	Kept the loop-based calculations because they satisfy the assignment requirements

Test evidence

Test type	Input	Expected result	Actual result	Status
Normal	100, 200, 300, 400, 500, 600, 700	Discount $15.00; total $135.00	Discount $15.00; total $135.00	Pass
Equal Values	500, 500, 500, 500, 500, 500, 500	Discount $10.00; total $90.00	Discount $10.00; total $90.00	Pass
Invalid input	Enter `abc` for one day, then correct it to `100`; remaining values: `200, 300, 400, 500, 600, 700`	Discount $0.00; total $99.99	Discount $0.00; total $99.99	Pass

Copied console output

```
Enter sales for day 2: 200
Enter sales for day 3: 300
Enter sales for day 4: 400
Enter sales for day 5: 500
Enter sales for day 6: 600
Enter sales for day 7: 700

Daily sales values:
Day 1: $100.00
Day 2: $200.00
Day 3: $300.00
Day 4: $400.00
Day 5: $500.00
Day 6: $600.00
Day 7: $700.00

Total sales: $2,800.00
Average sales: $400.00
```

```
Highest sales: $700.00
Lowest sales: $100.00
Number of days above average: 3
Week classification: Balanced

Enter sales for day 2: 500
Enter sales for day 3: 500
Enter sales for day 4: 500
Enter sales for day 5: 500
Enter sales for day 6: 500
Enter sales for day 7: 500

Daily sales values:
Day 1: $500.00
Day 2: $500.00
Day 3: $500.00
Day 4: $500.00
Day 5: $500.00
Day 6: $500.00
Day 7: $500.00

Total sales: $3,500.00
Average sales: $500.00
Highest sales: $500.00
Lowest sales: $500.00
Number of days above average: 0
Week classification: Concentrated

Enter sales for day 2: 200
Enter sales for day 3: 300
Enter sales for day 4: 400
Enter sales for day 5: 500
Enter sales for day 6: 600
Enter sales for day 7: 700

Daily sales values:
Day 1: $100.00
Day 2: $200.00
Day 3: $300.00
Day 4: $400.00
Day 5: $500.00
Day 6: $600.00
Day 7: $700.00

Total sales: $2,800.00
Average sales: $400.00
Highest sales: $700.00
Lowest sales: $100.00
Number of days above average: 3
Week classification: Balanced
```